5. Ejercicios de práctica

Instrucciones: Resuelve cada ejercicio mediante una expresión regular en Python. No se permite sustituir la expresión regular por una serie de condiciones manuales para revisar carácter por carácter. Escribe tu código en el espacio provisto.

5.1. Nivel 1. Básico: caracteres y cadenas

Ejercicio 1. Validación de letras. Acepta cadenas compuestas únicamente por letras del alfabeto inglés, sin espacios ni números.

Ejercicio 2. Validación de números enteros positivos. Acepta cadenas que contengan únicamente dígitos del 0 al 9.

Ejercicio 3. Cadena alfanumérica. Valida cadenas que contengan únicamente letras y números.

Ejercicio 4. Longitud exacta. Valida una cadena que tenga exactamente ocho caracteres (los espacios cuentan).

Ejercicio 5. Contraseña básica. Valida contraseñas de entre seis y doce caracteres, solo letras y números.

Ejercicio 6. Solo vocales minúsculas. Valida cadenas que contengan únicamente vocales minúsculas (a, e, i, o, u).

Ejercicio 7. Inicio y fin específicos. Valida cadenas que inicien con una letra mayúscula y terminen con un punto.

Ejercicio 8. Palabras de 5 letras. Valida cadenas que sean exactamente una palabra de 5 letras minúsculas.

Ejercicio 9. Alternancia simple. Valida cadenas que contengan exactamente la palabra "hola" o "adiós".

Ejercicio 10. Números con restricción. Valida números de exactamente 3 dígitos que inicien con el número 9.

Ejercicio 11. Exclusión de carácter. Valida cadenas que no contengan la letra "a" (ni mayúscula ni minúscula).

Ejercicio 12. Identificador de variable. Valida nombres de variable que inicien con letra o guion bajo, seguidos de alfanuméricos.

Ejercicio 13. Código RGB simple. Valida códigos de color hexadecimales de 3 caracteres (ej. `F0A`, `1b2`).

Ejercicio 14. Terminación de palabras. Valida palabras que terminen exactamente en "ción" o "sión".

Ejercicio 15. Un solo carácter. Valida cadenas que contengan estrictamente un solo carácter (ni vacío, ni más de uno).

5.2. Nivel 2. Intermedio: formatos y patrones

Ejercicio 16. Nombre de usuario. De 5 a 15 caracteres, inicia con letra, permite letras, números y guion bajo.

Ejercicio 17. Código postal mexicano. Exactamente cinco dígitos.

Ejercicio 18. Número telefónico. Exactamente diez dígitos, sin espacios ni separadores.

Ejercicio 19. Fecha básica. Formato DD/MM/AAAA (Día 01-31, Mes 01-12, Año 4 dígitos).

Ejercicio 20. Correo electrónico básico. Estructura general: usuario, @, dominio, extensión.

Ejercicio 21. Hora en formato 24h. Valida horas en formato HH:MM (00:00 a 23:59).

Ejercicio 22. Placa de automóvil. Formato de 3 letras mayúsculas seguidas de 3 dígitos (ej. `ABC123`).

Ejercicio 23. Dirección MAC. Valida formato XX:XX:XX:XX:XX:XX (hexadecimal).

Ejercicio 24. Nombre de dominio. Valida dominios simples (ej. `ejemplo.com`, `mi-sitio.org`).

Ejercicio 25. Número de seguridad social. Formato XXX-XX-XXXX (solo dígitos y guiones).

Ejercicio 26. Código de producto. Formato: 2 letras, 4 números, 1 letra (ej. `AB1234C`).

Ejercicio 27. Coordenadas simples. Valida formato básico de latitud y longitud (ej. `40.7128, -74.0060`).

Ejercicio 28. Formato de moneda. Símbolo $ o €, seguido de números y exactamente 2 decimales (ej. `$1,234.56`).

Ejercicio 29. Código IATA. Valida códigos de aeropuerto de exactamente 3 letras mayúsculas.

Ejercicio 30. ISBN-10 básico. Valida el formato con guiones (ej. `X-XXX-XXXXX-X`).

5.3. Nivel 3. Intermedio-avanzado: validaciones complejas

Ejercicio 31. Contraseña con mayúscula y minúscula. Solo letras, al menos una mayúscula y una minúscula.

Ejercicio 32. Contraseña con letras y números. Mínimo 8 caracteres, 1 mayúscula, 1 minúscula, 1 número. Sin símbolos.

Ejercicio 33. Contraseña robusta. Mínimo 8 caracteres, 1 mayúscula, 1 minúscula, 1 número, 1 símbolo especial.

Ejercicio 34. Número decimal. Enteros o decimales positivos/negativos. Signo opcional, punto decimal opcional.

Ejercicio 35. Número hexadecimal. Prefijo `0x` o `0X`, seguido de caracteres hexadecimales.

Ejercicio 36. Contraseña con 2 números. Mínimo 8 caracteres, al menos 2 números (no necesariamente consecutivos).

Ejercicio 37. Contraseña sin espacios. Mínimo 8 caracteres, no permite ningún tipo de espacio en blanco.

Ejercicio 38. Tarjeta de crédito. 16 dígitos agrupados de 4 en 4, separados por guiones o espacios.

Ejercicio 39. Notación científica. Valida números en notación científica (ej. `1.23e-4`, `-5E+10`).

Ejercicio 40. RFC mexicano (básico). 4 letras, 6 dígitos (AAMMDD), 3 caracteres alfanuméricos (homoclave).

Ejercicio 41. Expresión booleana simple. Valida variables de una letra unidas por "and" u "or" (ej. `A and B`).

Ejercicio 42. Ruta de archivo Windows. Valida rutas absolutas (ej. `C:\folder\file.txt`).

Ejercicio 43. Ruta de archivo Unix/Linux. Valida rutas absolutas (ej. `/usr/local/bin/script`).

Ejercicio 44. Teléfono con código de país. Formato internacional (ej. `+52 55 1234 5678`).

Ejercicio 45. Contraseña 3 de 4 categorías. Mínimo 8 chars. Debe cumplir al menos 3 de: mayúscula, minúscula,

5.4. Nivel 4. Avanzado: extracción y transformación

Ejercicio 46. Extracción de palabras. Extrae todas las palabras de una oración en español (incluye acentos y ñ).

Ejercicio 47. Extracción de números. Extrae todas las secuencias de dígitos de un texto mixto y guárdalas en lista.

Ejercicio 48. Normalización de espacios. Reemplaza secuencias de espacios, tabs o saltos de línea por un único espacio.

Ejercicio 49. Dirección IPv4. Valida 4 octetos (0-255) separados por puntos. Documenta si aceptas ceros a la izquierda.

Ejercicio 50. Extracción de etiquetas HTML. Extrae nombres de etiquetas de apertura. Explica las limitaciones con HTML anidado.

Ejercicio 51. Extraer correos de un texto. Dado un párrafo largo, extrae todas las direcciones de correo electrónico.

Ejercicio 52. Extraer Hashtags. Extrae todas las palabras precedidas por el símbolo # de un texto.

Ejercicio 53. Extraer Menciones. Extrae todas las menciones de usuario (inician con `@`) de un texto.

Ejercicio 54. Reemplazo de vocales. Reemplaza todas las vocales (con y sin acento) de un texto por un asterisco $*$.

Ejercicio 55. Extraer enlaces HTTP. Extrae todas las URLs que inicien con `http://` o `https://`.

Ejercicio 56. Dividir por comas (CSV). Divide una cadena por comas, pero ignora las comas que estén dentro de comillas dobles.

Ejercicio 57. Extraer fechas ISO. Extrae fechas en formato `YYYY-MM-DD` de un texto desordenado.

Ejercicio 58. Limpieza de texto no-ASCII. Elimina de una cadena cualquier carácter que no pertenezca al rango ASCII estándar.

Ejercicio 59. Extraer contenido de comillas. Extrae únicamente el texto que se encuentra dentro de comillas dobles.

5.5. Nivel 5. Experto: grupos, aserciones y backreferences

Ejercicio 61. Matrícula. 3 letras mayúsculas, guion, 4 dígitos. Usa grupos de captura para separar letras y números.

Ejercicio 62. Cadena sin espacios. Valida cadenas sin espacios en blanco. Rechaza cadenas vacías.

Ejercicio 63. Contraseña sin "admin". Mínimo 8 caracteres, no contiene la palabra "admin" (insensible a mayúsculas). Usa lookahead negativo.

Ejercicio 64. Extracción de precios. Extrae números precedidos por `$`. Usa lookbehind positivo.

Ejercicio 65. Grupos con nombre. Valida `Nombre:Juan;Edad:25`. Usa grupos con nombre `(?P<nombre>...)` para extraer ambos.

Ejercicio 66. Palabras entre comillas simples. Extrae palabras que estén estrictamente entre comillas simples, sin incluir las comillas (lookarounds).

Ejercicio 67. Contraseña sin el nombre de usuario. Valida contraseña que no contenga el nombre de usuario (usa lookahead negativo y variables de Python).

Ejercicio 68. Extraer dominio de correo. Extrae solo el dominio (después del @) de un correo usando lookbehind y lookahead.

Ejercicio 69. Palabras duplicadas. Encuentra palabras duplicadas consecutivas (ej. "el el") usando backreferences (`\1`).

Ejercicio 70. Números sin signo menos. Extrae números que NO estén precedidos por un signo menos (lookbehind negativo).

Ejercicio 71. Mismo carácter al inicio y fin. Valida cadenas que inician y terminan con el mismo carácter (usa backreference).

Ejercicio 72. Reformatear fechas. Dada una fecha `DD/MM/AAAA`, usa `re.sub()` y grupos para convertirla a `AAAA-MM-DD`.

Ejercicio 73. Extraer valores de JSON simple. Extrae los valores de un JSON simple (ej. `çlave": "valor"`), capturando solo el valor.